



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Розробка мобільних застосунків під Android»
Лабораторна робота №3
«Дослідження способів збереження даних»

Виконав:

Студент III курсу
гр. ІІІ-33
Соколов О. В.

Прийняв:

Орленко С. П.
“1” квітня 2026 р.

ЛАБОРАТОРНА РОБОТА №3

Тема: Дослідження способів збереження даних

Мета: дослідити способи збереження даних (база даних, файлова система, тощо) та отримати практичні навички щодо використання сховищ даних.

Завдання:

Написати програму під платформу Андроїд, яка доповнює програму, що розроблена за лабораторною роботою 2, роботою зі сховищами. Тобто при натисканні на кнопку «ОК» додатково:

- здійснюється запис результату взаємодії з інтерфейсом до сховища (файл або базу даних);

- користувач інформується відповідним повідомленням щодо успішності запису.

Також інтерфейс необхідно доповнити кнопкою «Відкрити», натискання на яку призводить до переходу на іншу Діяльність, у якій відображається вміст даних, що зберігаються у сховищі. Якщо дані відсутні (сховище порожнє) відобразити відповідне повідомлення. За бажанням можна додатково реалізувати оновлення та видалення даних зі сховища.

РОБОТА ПРОГРАМИ

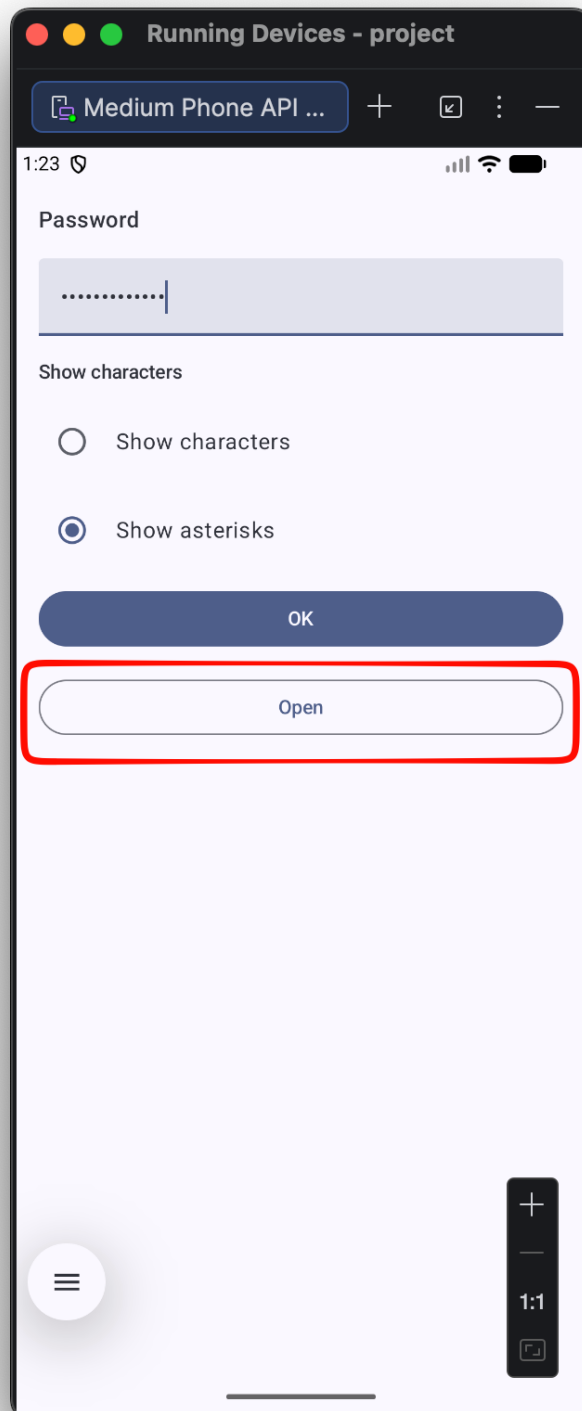


Рис. 1 – Вікно програми після запуску із новою кнопкою Open

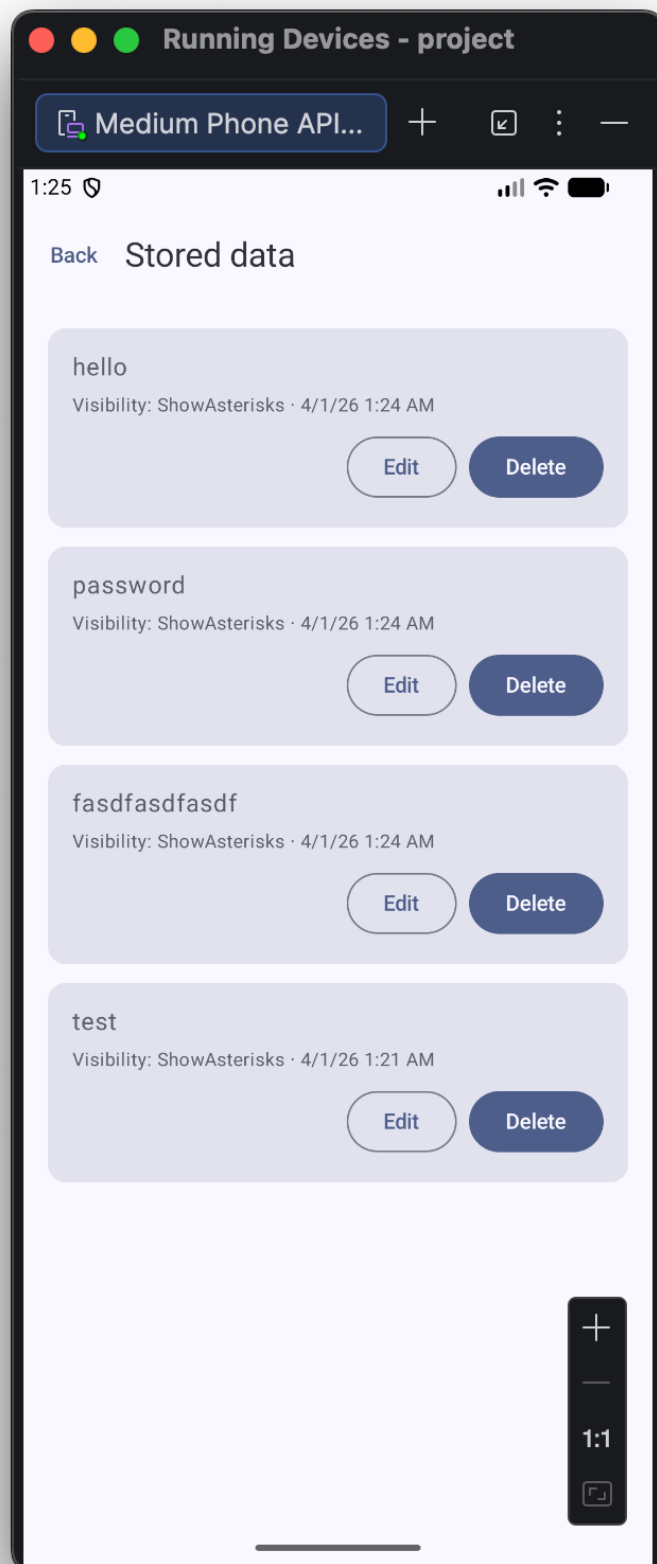


Рис. 2 – Збережені паролі

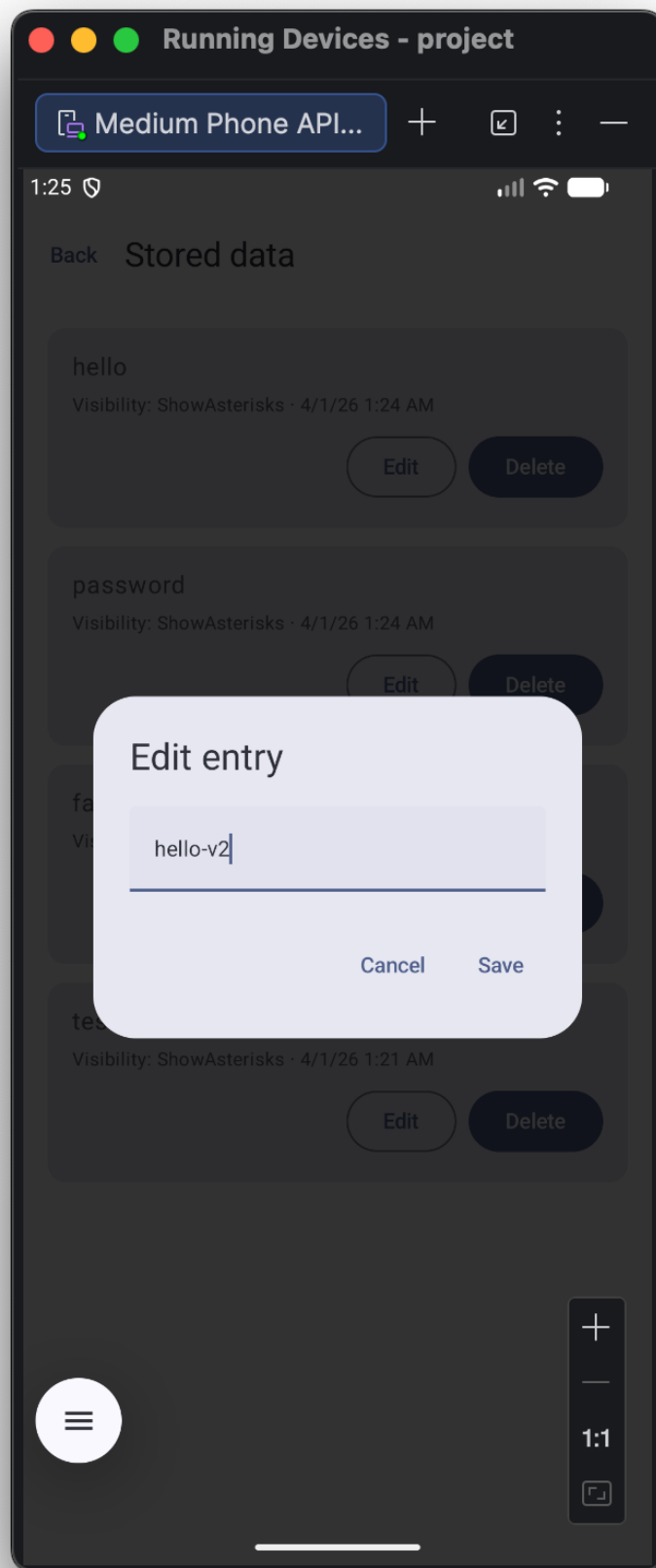


Рис. 3 – Модалка редагування збереженого запису

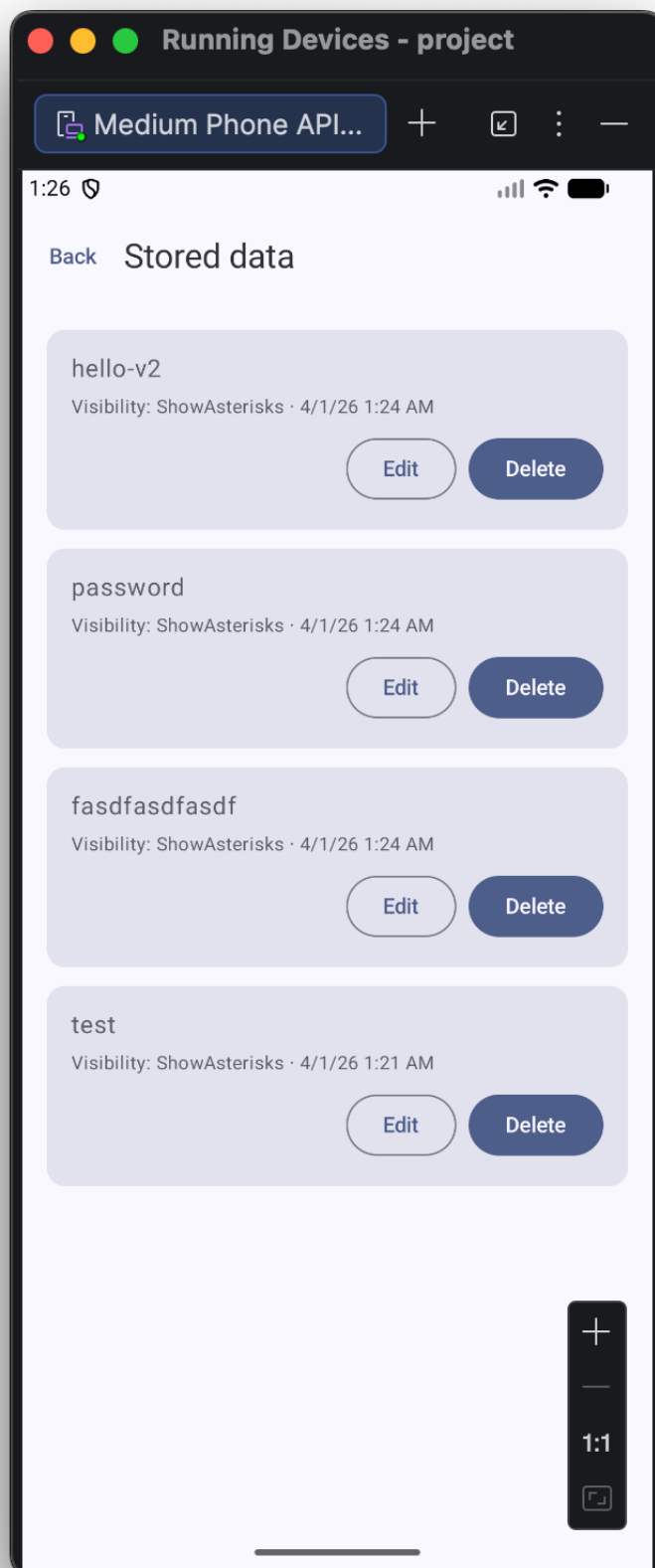


Рис. 4 – Відредагований перший запис

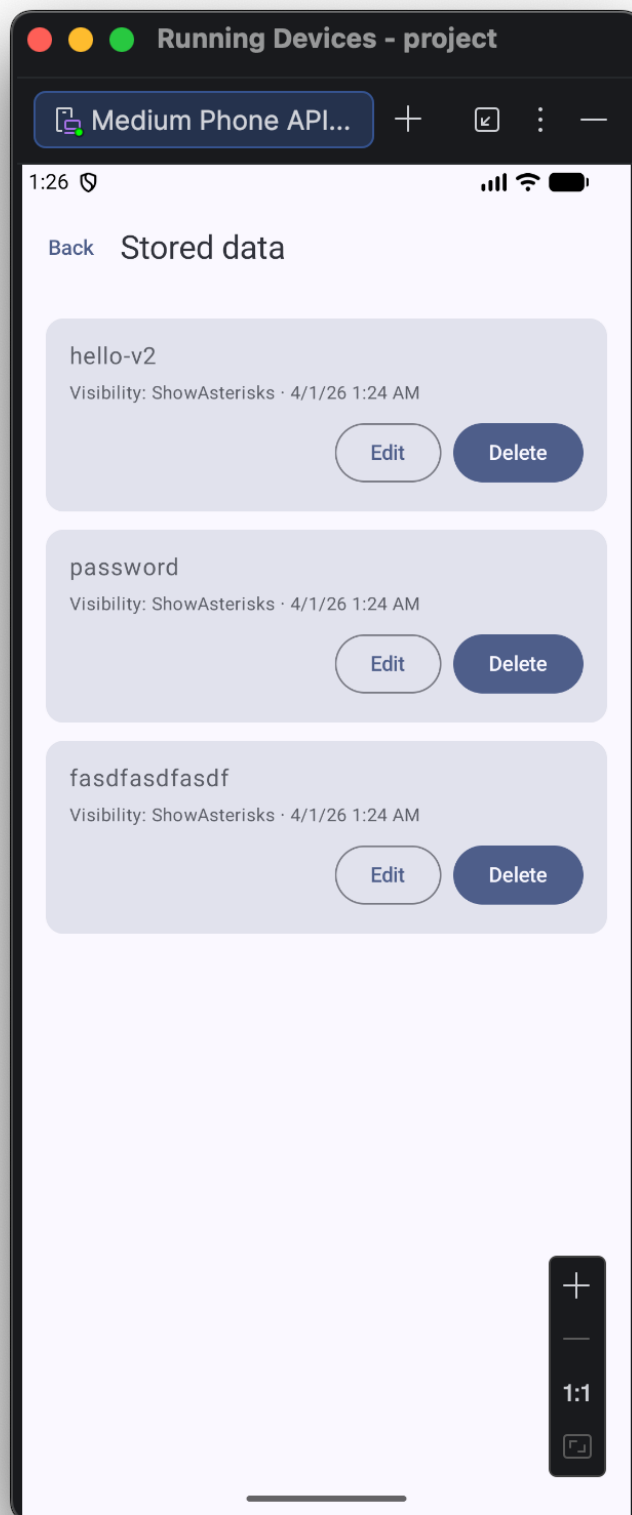


Рис. 5 – Видалений останній запис

КОНТРОЛЬНІ ПИТАННЯ

1. У Android для цього використовують SharedPreferences (або DataStore як сучасну заміну): дані зберігаються у вигляді набору пар ключ–значення в XML-файлі в приватній директорії застосунку. Читання й запис виконують через об’єкт SharedPreferences та редактор (edit()), типи значень обмежені (рядок, число, булеве тощо). Підходить для невеликих налаштувань користувача, прапорців і простих параметрів, а не для великих або структурованих даних.

2. Внутрішнє сховище – лише для вашого застосунку, дані недоступні іншим (безпека, кеш, БД). Зовнішнє (спільне) – для файлів, якими можна ділитися з іншими застосунками (медіа, документи), часто потрібні дозволи або Storage Access Framework. Кеш – тимчасові дані для прискорення роботи; система може очистити при нестачі місця. Асети – файли, упаковані в APK, лише для читання (ресурси, шаблони).

3. Застосунок отримує контекст і через нього – шляхи до каталогів (filesDir, cacheDir, за потреби зовнішнього сховища). Файл відкривають для читання/запису (поток, при потребі – з буфером), після роботи поток закривають. Створюють каталоги, перевіряють наявність місця та помилки доступу. Для Android 10+ зовнішнє сховище часто обмежують scoped storage – доступ через URI та системні засоби, а не довільні шляхи.

4. SQLiteOpenHelper – клас, який створює базу при першому зверненні та оновлює схему при зміні версії (onCreate, onUpgrade). Розробник описує SQL-таблиці, відкриває SQLiteDatabase, виконує запити (rawQuery, insert, update, delete) або використовує ContentValues. Переваги: повний контроль, нативна підтримка SQLite, без додаткових бібліотек. Недоліки: багато ручного коду, легко помилитися в SQL і міграціях, немає перевірки типів на етапі компіляції, більше коду для потоків і життєвого циклу.

5. Room: робота з БД, переваги та недоліки. Room – обгортка над SQLite: описують сутності (@Entity), доступ (@Dao) і базу (@Database).

Компілятор генерує реалізацію запитів; можна використовувати корутини, Flow для реактивних списків. Переваги: менше помилок у SQL, перевірки на етапі збірки, зручні міграції, інтеграція з Kotlin/Android. Недоліки: потрібно навчитися анотацій і обмежень Room, для дуже складних запитів інколи пишуть сирий SQL; збільшується розмір залежностей і час збірки.

6. Діагональ (дюйми), роздільна здатність (пікселі), щільність (dpi), співвідношення сторін, яскравість і контрастність, частота оновлення, тип матриці (IPS, OLED тощо), споживання енергії, наявність сенсорного вводу, інколи – згинання, отвори під камеру, вирізи.

7. За технологією матриці: LCD (TN, IPS), OLED/AMOLED (глибший чорний, гнучкі панелі). За форм-фактором: плоскі, зігнуті по краю, складані. За призначенням: основний дисплей, додатковий зовнішній на «книжці». Відрізняються якістю кольору, кутами огляду, енергоспоживанням і вартістю виробництва.

8. Сенсорний екран – дисплей, який одночасно показує зображення і реагує на дотик (координати, жести). Характеристики: кількість одночасних дотиків (мультитач), чутливість, точність, швидкість відгуку, підтримка стилуса (деякі пристрої), покриття (олеофобне тощо).

9. За технологією визначення координат: резистивні (тиск шарами, простіші, гірша якість) і ємнісні (найпоширеніші в смартфонах). Серед ємнісних виділяють поверхневі та проєкційно-ємнісні (багатоточкові жести). Є також інфрачервоні сітки, SAW тощо – рідше в масових телефонах.

10. Збільшувати зони натискання (кнопки, відступи), уникати дрібного тексту й щільних елементів, передбачати жести (прокрутка, свайп) замість лише дрібних кліків. Враховувати великий палець і зручність однією рукою, давати візуальний відгук (анімація, стан натискання). Тестувати на різних розмірах екранів; дотримуватися Material Design / HIG для передбачуваності та доступності.

ВИСНОВОК

У ході виконання лабораторної роботи було розроблено мобільний застосунок для платформи Android на основі програми з попередньої роботи: збережено екран введення паролю з радіо-кнопками режиму відображення та екран результату з кнопкою «Cancel», а також додано збереження результату взаємодії з інтерфейсом у локальну базу даних за допомогою бібліотеки Room, інформування користувача про успішність запису через Toast та окрему другу активність з переглядом збережених записів за кнопкою «Відкрити» (Open). Реалізовано відображення порожнього сховища, список записів, а також оновлення та видалення даних. Інтерфейс головного сценарію побудовано в Jetpack Compose з Navigation у межах однієї активності та ViewModel для стану; доступ до бази організовано через підклас Application і AndroidViewModel. Таким чином, було закріплено навички роботи зі сховищами даних на Android, локальною БД SQLite через Room, другою активністю та реактивним оновленням списку даних.

КОД ДОДАТКУ

Посилання на GitHub репозиторій:

<https://github.com/sokolovgit/android-mobile-development>